

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import accuracy_score, classification_report, confusion_matri

```

Loading data

```

1 df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")
2
3 df.head()

```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educa
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Lif
1	49	No	Travel_Frequently	279	Research & Development	8	1	Lif
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Lif
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 35 columns

```

1 print("Rows and Columns:", df.shape)
2 print("\nColumn Names:")
3 print(df.columns)
4
5 print("\nMissing Values:")
6 print(df.isnull().sum())

```

Rows and Columns: (1470, 35)

Column Names:

```

Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
      'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',
      'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',
      'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
      'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
      'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',
      'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',
      'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',
      'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
      'YearsWithCurrManager'],
      dtype='object')

```

Missing Values:

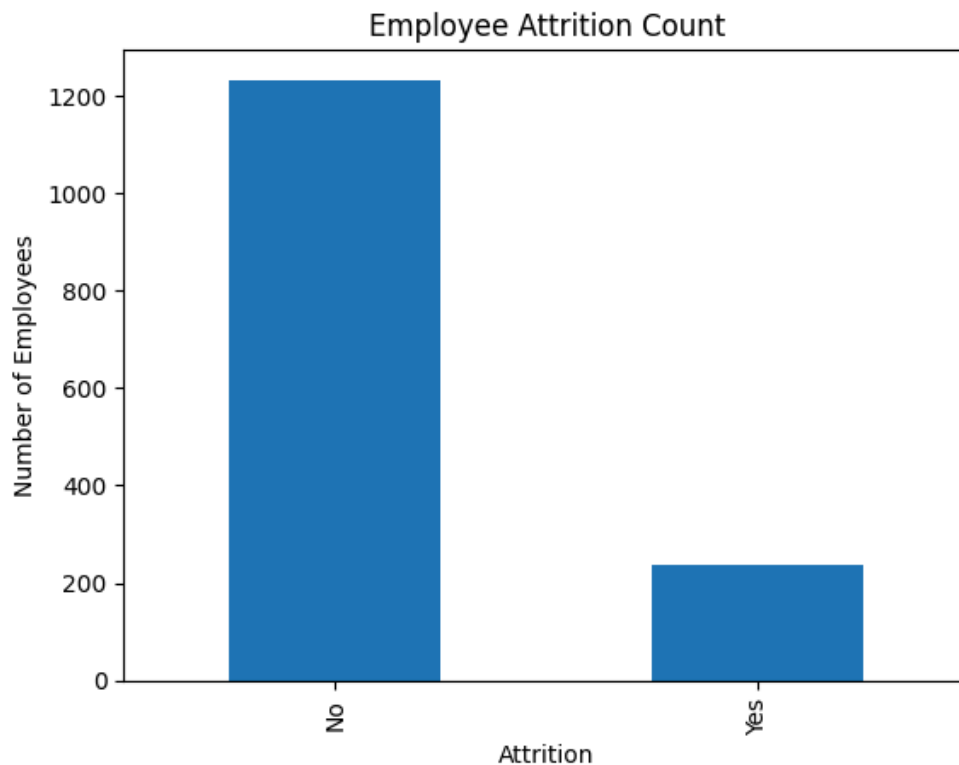
Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0

```
Education 0
EducationField 0
EmployeeCount 0
EmployeeNumber 0
EnvironmentSatisfaction 0
Gender 0
HourlyRate 0
JobInvolvement 0
JobLevel 0
JobRole 0
JobSatisfaction 0
MaritalStatus 0
MonthlyIncome 0
MonthlyRate 0
NumCompaniesWorked 0
Over18 0
OverTime 0
PercentSalaryHike 0
PerformanceRating 0
RelationshipSatisfaction 0
StandardHours 0
StockOptionLevel 0
TotalWorkingYears 0
TrainingTimesLastYear 0
WorkLifeBalance 0
YearsAtCompany 0
YearsInCurrentRole 0
YearsSinceLastPromotion 0
YearsWithCurrManager 0
dtype: int64
```

Attrition count

```
1 print(df["Attrition"].value_counts())
2
3 df["Attrition"].value_counts().plot(kind="bar")
4 plt.title("Employee Attrition Count")
5 plt.xlabel("Attrition")
6 plt.ylabel("Number of Employees")
7 plt.show()
```

```
Attrition
No      1233
Yes      237
Name: count, dtype: int64
```



Encoding text columns

```
1 le = LabelEncoder()
2
3 for column in df.columns:
4     if df[column].dtype == "object":
5         df[column] = le.fit_transform(df[column])
6
7 df.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Education- Years
0	41	1	2	1102	2	1	2	
1	49	0	1	279	1	8	1	
2	37	1	2	1373	1	2	2	
3	33	0	1	1392	1	3	4	
4	27	0	2	591	1	2	1	

5 rows × 35 columns

Training model

```
1 X = df.drop(["Attrition", "EmployeeNumber", "EmployeeCount", "StandardHours"],
2             axis=1)
3 y = df["Attrition"]
4
5 X_train, X_test, y_train, y_test = train_test_split(
```

```

6     X, y, test_size=0.2, random_state=42
7 )
8
9     model = RandomForestClassifier(random_state=42)
10    model.fit(X_train, y_train)
11
12    y_pred = model.predict(X_test)

```

Accuracy

```

1    accuracy = accuracy_score(y_test, y_pred)
2
3    print("Model Accuracy:", round(accruacy * 100, 2), "%")
4    print("\nClassification Report:")
5    print(classification_report(y_test, y_pred))

```

Model Accuracy: 88.1 %

Classification Report:

	precision	recall	f1-score	support
0	0.88	1.00	0.94	255
1	1.00	0.10	0.19	39
accuracy			0.88	294
macro avg	0.94	0.55	0.56	294
weighted avg	0.90	0.88	0.84	294

Although the model achieved 88.10% accuracy, recall for employees who left is low due to class imbalance. Future improvement can include SMOTE, class weights, or XGBoost.

Feature importance

```

1    importance = pd.DataFrame({
2        "Feature": X.columns,
3        "Importance": model.feature_importances_
4    })
5
6    importance = importance.sort_values(by="Importance", ascending=False)
7
8    print(importance.head(10))

```

	Feature	Importance
15	MonthlyIncome	0.084692
19	OverTime	0.063224
0	Age	0.059147
2	DailyRate	0.053481
16	MonthlyRate	0.051434
24	TotalWorkingYears	0.051000
9	HourlyRate	0.047514
4	DistanceFromHome	0.047094
27	YearsAtCompany	0.043692
17	NumCompaniesWorked	0.035327

Top 10 Important Features Chart

```

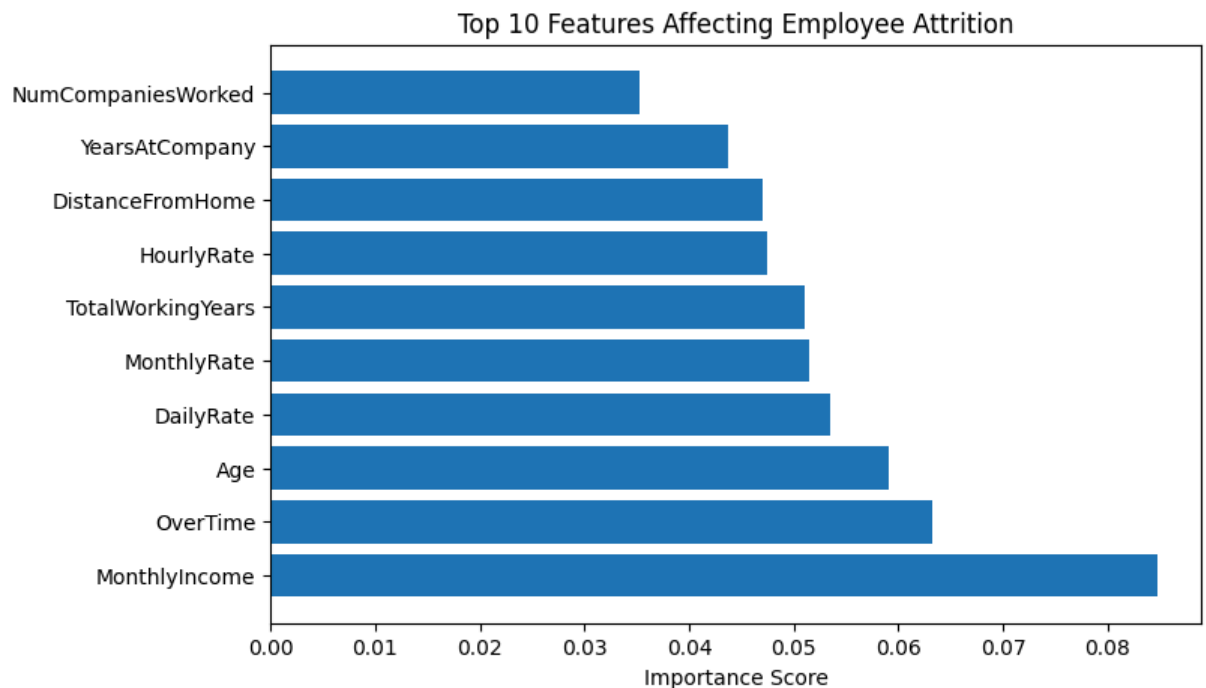
1    import matplotlib.pyplot as plt
2
3    top10 = importance.head(10)

```

```

4
5 plt.figure(figsize=(8,5))
6 plt.barh(top10["Feature"], top10["Importance"])
7 plt.title("Top 10 Features Affecting Employee Attrition")
8 plt.xlabel("Importance Score")
9 plt.show()

```



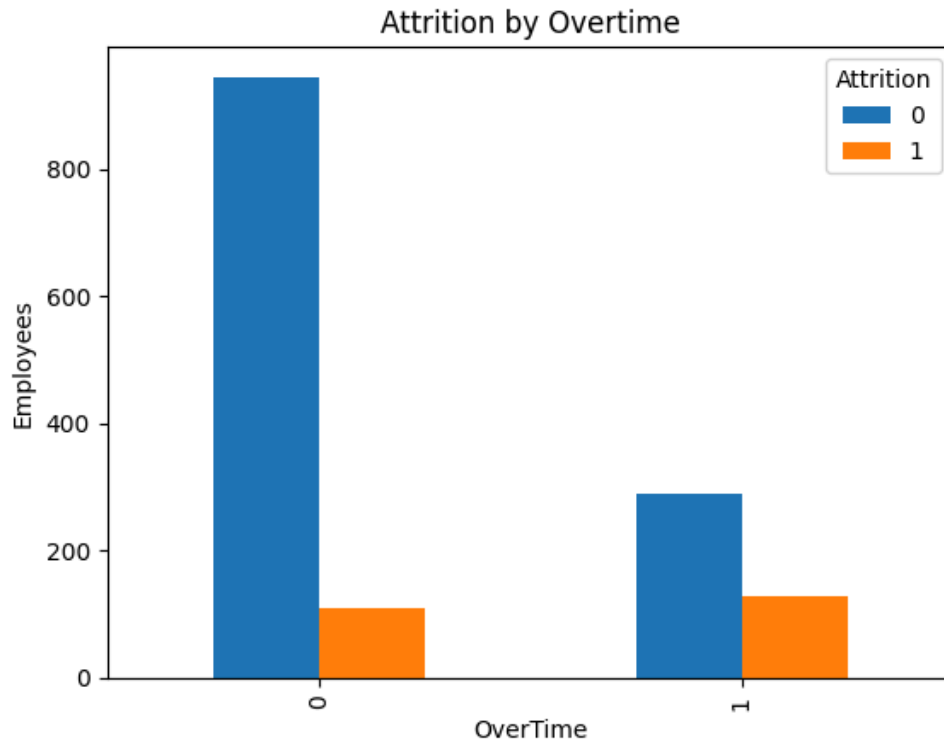
Attrition by Overtime

```

1 import pandas as pd
2
3 overtime_attrition = pd.crosstab(
4     df["OverTime"],
5     df["Attrition"]
6 )
7
8 print(overtime_attrition)
9
10 overtime_attrition.plot(kind="bar")
11 plt.title("Attrition by Overtime")
12 plt.ylabel("Employees")
13 plt.show()

```

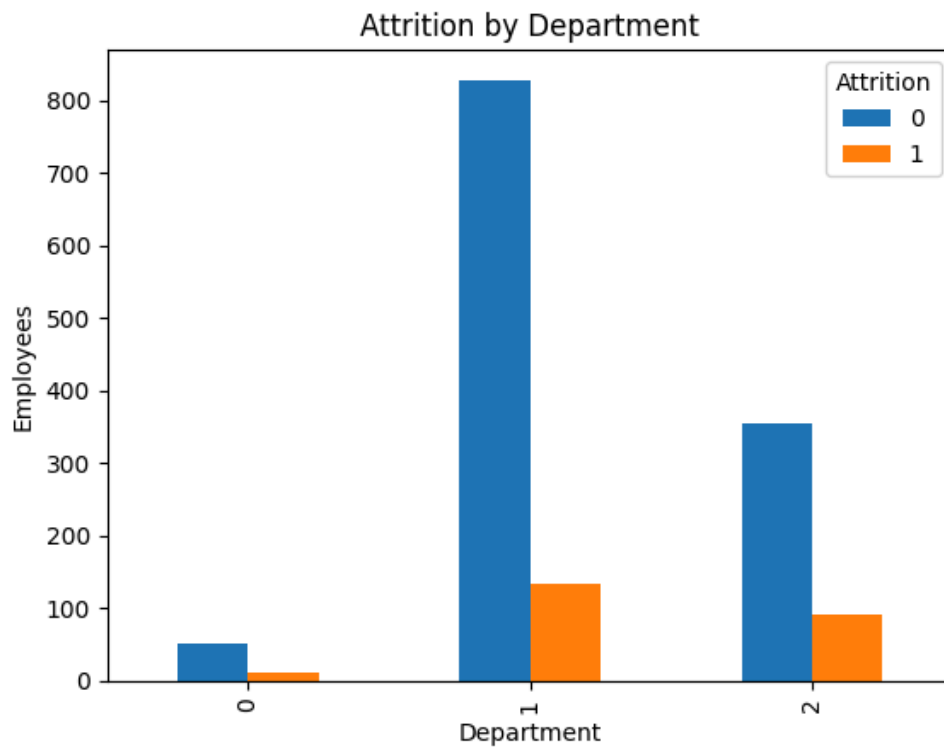
Attrition	0	1
OverTime		
0	944	110
1	289	127



Employees working overtime show higher attrition rates compared to employees who do not work overtime.

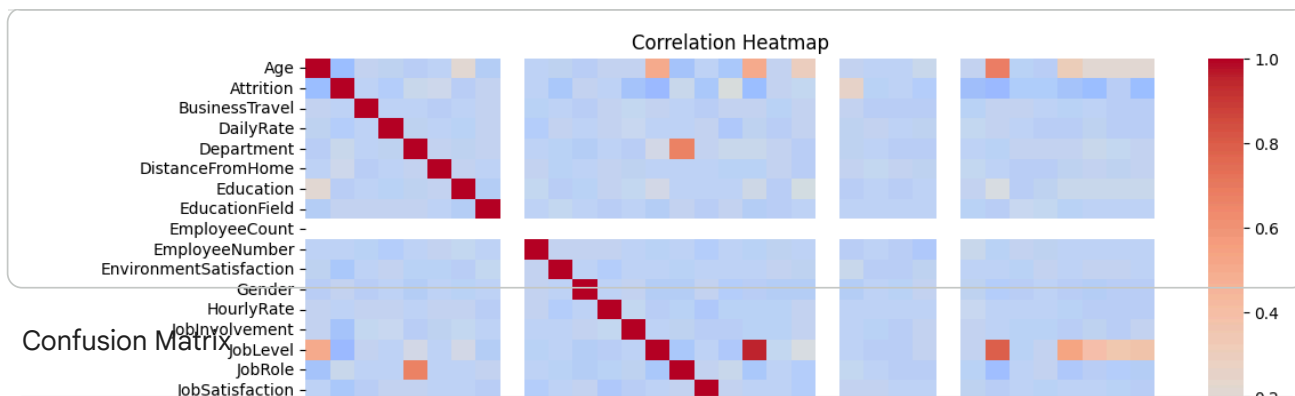
Attrition by Department

```
1 department_attrition = pd.crosstab(  
2     df["Department"],  
3     df["Attrition"]  
4 )  
5  
6 department_attrition.plot(kind="bar")  
7 plt.title("Attrition by Department")  
8 plt.ylabel("Employees")  
9 plt.show()
```



Correlation Heatmap

```
1 import seaborn as sns
2
3 plt.figure(figsize=(12,8))
4
5 sns.heatmap(
6     df.corr(),
7     cmap="coolwarm"
8 )
9
10 plt.title("Correlation Heatmap")
11 plt.show()
```



```

1 from sklearn.metrics import ConfusionMatrixDisplay
2
3 ConfusionMatrixDisplay.from_estimator(
4     model,
5     X_test,
6     y_test
7 )
8
9 plt.title("Confusion Matrix")
10 plt.show()

```

